Credit Card Fraud Detection
Parts 3, 4, and 5 Report

Contents

1 Part 3: Data Preprocessing

Data preprocessing is the process of cleaning and transforming raw data into a format suitable for machine learning models. Models only understand numbers, so all text, dates, and inconsistencies must be resolved before training.

1.1 Step 1: Loading the Dataset

```
df = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTrain.csv"))
df_test = pd.read_csv(os.path.join(SCRIPT_DIR, "fraudTest.csv"))
```

Output:

```
Train: 1,296,675 rows x 23 columns
Test:  555,719 rows x 23 columns
```

Both training and testing datasets are loaded separately. The same preprocessing steps are applied to both to ensure consistency.

1.2 Step 2: Handling Missing Values

```
missing = df.isnull().sum()
total_missing = missing.sum()
```

Output:

```
Total missing values: 0
-> No missing values found!
```

Explanation:

- `isnull().sum()` counts missing values in each column.
- If missing values existed, numerical columns would be filled using `mean()`, and categorical columns using `mode()` (most frequent value).
- This dataset has no missing values.

1.3 Step 3: Removing Duplicates

```
duplicates = df.duplicated().sum()
df = df.drop_duplicates().reset_index(drop=True)
```

Output:

```
Duplicate rows found: 0
-> No duplicates found!
```

Duplicate rows can bias the model by overrepresenting certain patterns. The function `drop_duplicates()` removes exact duplicate rows.

1.4 Step 4: Dropping Irrelevant Columns

```python
drop_cols = ["Unnamed: 0", "cc_num", "first", "last",
             "street", "trans_num", "unix_time"]
df = df.drop(columns=drop_cols)
```

Why each column was dropped:

Column	Reason for Dropping
Unnamed: 0	Auto-generated index, not a real feature
cc_num	Credit card number is an identifier, not predictive
first, last	Personal names have no predictive value
street	Too many unique addresses, not useful for modeling
trans_num	Unique transaction ID, not a feature
unix_time	Redundant with trans_date_trans_time

After dropping: shape changed from 23 to 16 columns.

1.5 Step 5: Date/Time Feature Engineering

```python
# Extract useful features from datetime
dataset["trans_hour"] = dataset["trans_date_trans_time"].dt.hour
dataset["trans_day"] = dataset["trans_date_trans_time"].dt.dayofweek
dataset["trans_month"] = dataset["trans_date_trans_time"].dt.month

# Calculate age from date of birth
dataset["age"] = (dataset["trans_date_trans_time"]
                  - dataset["dob"]).dt.days // 365
```

Explanation:

Date strings are not useful for ML models. Instead, we extract meaningful numerical features:

New Feature	What It Represents
trans_hour	Hour of transaction (0–23). Fraud increases at night.

trans_day	Day of week (0=Monday to 6=Sunday)
trans_month	Month of the year (1–12)
age	Customer age calculated from date of birth

After extraction, the original date columns (`trans_date_trans_time` and `dob`) are dropped. Final shape: 18 columns.

1.6 Step 6: Handling Outliers (Capping with IQR)

```
Q1 = df["amt"].quantile(0.25)
Q3 = df["amt"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df["amt"] = df["amt"].clip(lower=lower, upper=upper)
```

Output:

```
amt:      Capped 67,290 outliers (5.19%)
city_pop: Capped 242,674 outliers (18.72%)
```

Why capping instead of removing?

Removing outlier rows would delete too much data. Capping (clipping) keeps the rows but limits extreme values to the IQR-based bounds. This preserves data while reducing the influence of extreme values.

1.7 Step 7: Cleaning Text Data

```
dataset[col] = dataset[col].str.strip().str.lower()
```

- `strip()` removes leading and trailing whitespace.
- `lower()` converts all text to lowercase.
- Ensures consistency: "Shopping_NET" and "shopping_net" become the same value.

Applied to columns: merchant, category, city, state, job.

1.8 Step 8: Label Encoding

```python
# Binary encoding for gender
dataset["gender"] = dataset["gender"].map({"M": 0, "F": 1})

# LabelEncoder for other categorical columns
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["category"] = le.fit_transform(df["category"])
```

Output:

```
gender:   M->0, F->1 (Binary Encoding)
merchant: 693 unique values encoded
category: 14 unique values encoded
city:     906 unique values encoded
state:    51 unique values encoded
job:      497 unique values encoded

-> All categorical columns are now numerical!
   Numerical: 18 columns
   Object:    0 columns
```

Explanation:

Machine learning models require numerical input. LabelEncoder converts each unique text value to a number. For example:

```
grocery_pos   -> 5
gas_transport -> 4
shopping_net  -> 12
```

1.9 Step 9: Feature Scaling

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# StandardScaler: transforms to mean=0, std=1
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# MinMaxScaler: transforms to range [0, 1]
mm_scaler = MinMaxScaler()
X_train_minmax = mm_scaler.fit_transform(X_train)
```

Output:

```
StandardScaler applied!
  Example (amt): mean=0.0000, std=1.0000

MinMaxScaler example:
  Example (amt): min=0.0000, max=1.0000

-> Using StandardScaler for model training
   (better for KNN and SVM)
```

Why scaling is important:

Without scaling, features with large ranges (e.g., amt: 1–29000) dominate over features with small ranges (e.g., gender: 0–1). KNN calculates distances and SVM finds boundaries, so both are very sensitive to feature scales. StandardScaler was chosen because it works best with these algorithms.

1.10 Preprocessing Summary

```
Steps Applied:
[1] Missing Values    -> Checked (none found)
[2] Duplicates        -> Checked (none found)
[3] Irrelevant Cols   -> Dropped 7 columns
[4] Date/Time         -> Extracted hour, day, month, age
[5] Outliers          -> Capped using IQR method
[6] Text Cleaning     -> strip() + lower()
[7] Label Encoding    -> All categorical -> numerical
[8] Feature Scaling   -> StandardScaler applied

Final Dataset:
X_train shape: (1,296,675, 17) - 17 features
X_test shape:  (555,719, 17)
```

2 Part 4: Model Building

This section trains three machine learning models on the preprocessed data and compares their performance.

2.1 Subsampling Strategy

```
SAMPLE_SIZE = 30000   # Instead of 1,296,675
TEST_SAMPLE = 10000   # Instead of 555,719

X_train_sample, _, y_train_sample, _ = train_test_split(
    X_train_full, y_train_full,
    train_size=SAMPLE_SIZE,
    random_state=42,
    stratify=y_train_full  # Keep same fraud ratio
)
```

Why subsampling?

- KNN calculates distances to every training point for each prediction — very slow on 1.3M rows.
- SVM solves a complex optimization problem that scales poorly with large data.
- A stratified subsample of 30,000 rows preserves the original class ratio (0.58% fraud).

Output:

```
Train sample: (30000, 17)
Test sample:  (10000, 17)
Train fraud ratio: 0.58%
Test fraud ratio:  0.39%
```

2.2 Model 1: K-Nearest Neighbors (KNN)

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_sample, y_train_sample)
knn_pred = knn.predict(X_test_sample)
```

How KNN works:

For each new transaction, KNN finds the 5 closest transactions in the training data. The majority class among those 5 neighbors determines the prediction. If 4 out of 5 neighbors are "Not Fraud", KNN predicts "Not Fraud".

Output:

```
Training + Prediction time: 2.77 seconds

Accuracy:  0.9961
Precision: 0.0000
Recall:    0.0000
F1-Score:  0.0000

Confusion Matrix:
TN= 9961  FP=    0
FN=   39  TP=    0
```

Analysis: KNN predicted every single transaction as Not Fraud. It detected zero fraud cases. The 99.61% accuracy is completely misleading because the data is 99.4% non-fraud anyway.

2.3 Model 2: Support Vector Machine (SVM)

```
svm = SVC(kernel="rbf", random_state=42)
svm.fit(X_train_sample, y_train_sample)
svm_pred = svm.predict(X_test_sample)
```

How SVM works:

SVM finds the optimal decision boundary (hyperplane) that separates fraud from non-fraud with the maximum margin. The RBF kernel allows this boundary to be curved instead of a straight line.

Output:

```
Training + Prediction time: 4.26 seconds

Accuracy:  0.9961
Precision: 0.0000
Recall:    0.0000
F1-Score:  0.0000

Confusion Matrix:
TN= 9961  FP=    0
FN=   39  TP=    0
```

Analysis: Same problem as KNN. SVM also failed to detect any fraud due to the extreme class imbalance.

2.4 Model 3: Random Forest

```
rf = RandomForestClassifier(n_estimators=100, random_state=42,
                            n_jobs=-1)
rf.fit(X_train_sample, y_train_sample)
rf_pred = rf.predict(X_test_sample)
```

How Random Forest works:

Random Forest builds 100 independent decision trees. Each tree is trained on a random subset of data and features. The final prediction is determined by majority voting — whichever class gets the most votes from all 100 trees wins.

Output:

```
Training + Prediction time: 0.59 seconds

Accuracy:  0.9966
Precision: 1.0000
Recall:    0.1282
F1-Score:  0.2273

Confusion Matrix:
TN= 9961  FP=    0
FN=   34  TP=    5
```

Analysis: Random Forest detected 5 out of 39 fraud cases. Precision is perfect (every fraud prediction was correct), but recall is low (only 12.8% of actual fraud was caught).

2.5 Model Comparison

Model	Accuracy	Precision	Recall	F1	Time
KNN	0.9961	0.0000	0.0000	0.0000	2.77s
SVM	0.9961	0.0000	0.0000	0.0000	4.26s
Random Forest	0.9966	1.0000	0.1282	0.2273	0.59s

Key Findings:

1. All models achieved over 99% accuracy, but this is misleading with imbalanced data.
2. F1-Score is the correct metric because it balances Precision and Recall.
3. Random Forest is the best performer but still has low recall.
4. The core problem is that only 0.58% of transactions are fraud.
5. Models need class weighting to handle imbalance (addressed in Part 5).

2.6 Confusion Matrix Explained

Term	Meaning
TN (True Negative)	Correctly predicted as Not Fraud
FP (False Positive)	Incorrectly predicted as Fraud (false alarm)
FN (False Negative)	Fraud that was missed by the model
TP (True Positive)	Correctly detected as Fraud

Metric Formulas:

- Accuracy = (TP + TN) / Total
- Precision = TP / (TP + FP) — Of all fraud predictions, how many were correct?
- Recall = TP / (TP + FN) — Of all actual fraud, how many were detected?
- F1 = 2 * (Precision * Recall) / (Precision + Recall)

3 Part 5: Hyperparameter Tuning

Hyperparameters are settings that are chosen before training begins. The model does not learn them automatically. Tuning means systematically testing different settings to find the best combination.

3.1 Tools Used

Tool	Description
GridSearchCV	Tries every possible combination of parameters
RandomizedSearchCV	Tries a random subset of combinations (faster)
cv=3	3-fold cross-validation: splits data into 3 parts, trains on 2 and tests on 1, repeats 3 times
scoring="f1"	Optimizes for F1-Score instead of accuracy

3.2 Tuning KNN: Finding Best K

```
knn_params = {"n_neighbors": [3, 5, 7, 9, 11]}

knn_grid = GridSearchCV(
    KNeighborsClassifier(),
    knn_params,
    scoring="f1",
    cv=3,
    n_jobs=-1
)
knn_grid.fit(X_train, y_train)
```

What K means: The number of nearest neighbors used for voting. Smaller K is more sensitive to local patterns. Larger K is more stable but may miss rare cases like fraud.

Output:

```
K= 3  ->  F1 = 0.0328
K= 5  ->  F1 = 0.0000
K= 7  ->  F1 = 0.0000
K= 9  ->  F1 = 0.0000
K=11  ->  F1 = 0.0000

--> Best K = 3
--> Best CV F1 = 0.0328
```

```
Test Results:
Accuracy: 0.9959 | Precision: 0.0000
Recall:   0.0000 | F1-Score:  0.0000
TN=9959  FP=2  FN=39  TP=0
```

Analysis: KNN struggles with highly imbalanced data because fraud samples are always surrounded by overwhelming numbers of non-fraud neighbors. Even with the best K, it cannot detect fraud effectively.

3.3 Tuning SVM: Kernel, C, and Class Weight

```python
svm_params = {
    "kernel": ["rbf", "linear"],
    "C": [0.1, 1, 10],
    "class_weight": ["balanced"]  # Key for imbalance!
}

svm_grid = GridSearchCV(
    SVC(random_state=42),
    svm_params,
    scoring="f1",
    cv=3,
    n_jobs=-1
)
```

Parameter explanations:

Parameter	Meaning
kernel=“rbf”	Non-linear curved decision boundary. Better for complex patterns
kernel=“linear”	Straight-line decision boundary. Simpler but faster
C=0.1, 1, 10	Regularization strength. Higher C means the model tries harder to classify every point correctly (risk of overfitting)
class_weight=“balanced”	Automatically adjusts weights inversely proportional to class frequency. Fraud gets a weight of approximately 171x compared to non-fraud

Output:

```
C=0.1 + rbf     + balanced -> F1 = 0.1477
```

```
C=0.1 + linear + balanced -> F1 = 0.0726
C=1   + rbf    + balanced -> F1 = 0.2199
C=1   + linear + balanced -> F1 = 0.0726
C=10  + rbf    + balanced -> F1 = 0.2245 (Best)
C=10  + linear + balanced -> F1 = 0.0726

--> Best: kernel=rbf, C=10, class_weight=balanced

Test Results:
Accuracy:  0.9937
Precision: 0.1842
Recall:    0.1795
F1-Score:  0.1818

Confusion Matrix:
TN=9930  FP=31  FN=32  TP=7
```

Key Improvement: SVM now detects 7 fraud cases instead of 0. The `class_weight="balanced"` parameter was the critical change. It tells SVM that misclassifying a fraud transaction is 171 times worse than misclassifying a normal transaction.

3.4 Tuning Random Forest

```python
rf_params = {
    "n_estimators": [50, 100, 200],
    "max_depth": [10, 20, None],
    "class_weight": ["balanced"]
}

rf_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    rf_params,
    n_iter=6,
    scoring="f1",
    cv=3,
    random_state=42
)
```

Parameter explanations:

Parameter	Meaning

n_estimators	Number of trees in the forest. More trees = better but slower
max_depth=10	Maximum depth of each tree. Limits complexity to prevent overfitting
max_depth=None	No limit, trees grow until fully expanded
class_weight="balanced"	Same as SVM, gives more importance to the fraud class

Output:

```
100 trees + depth=None + balanced -> F1 = 0.1478
100 trees + depth=10   + balanced -> F1 = 0.3011
200 trees + depth=20   + balanced -> F1 = 0.1823
50 trees  + depth=10   + balanced -> F1 = 0.2919
200 trees + depth=None + balanced -> F1 = 0.1564
200 trees + depth=10   + balanced -> F1 = 0.3053 (Best)

--> Best: n_estimators=200, max_depth=10

Test Results:
Accuracy:  0.9874
Precision: 0.1282
Recall:    0.3846
F1-Score:  0.1923

Confusion Matrix:
TN=9859  FP=102  FN=24  TP=15
```

Key Improvement: Random Forest now detects 15 fraud cases instead of 5. Recall improved from 12.8% to 38.5%. The `max_depth=10` prevents overfitting by limiting how complex each tree can become.

3.5 Before vs After Tuning Comparison

Model	Before F1	After F1	TP Before	TP After
KNN	0.0000	0.0000	0	0
SVM	0.0000	0.1818	0	7
Random Forest	0.2273	0.1923	5	15

3.6 Key Takeaways from Tuning

1. **class_weight="balanced"** is the most impactful change. It addresses class imbalance by telling the model that misclassifying fraud is much more costly.
2. **GridSearchCV** systematically finds the best parameter combination using cross-validation.
3. **F1-Score** is the correct optimization metric for imbalanced datasets, not accuracy.
4. Random Forest with tuning detects **3 times more fraud** (15 vs 5).
5. SVM went from detecting **zero fraud to 7 cases** after adding class weights.
6. KNN still struggles because it has no built-in class weighting mechanism.